# 4 接口测试笔记

## 接口测试基础

### 1 相关概念

#### 1.1 接口介绍

**软件接口的由来**

- 软件系统中, **前端**和**后端**是两大重要组成部分
- **前端**主要用于**与用户交互**, 用户通过前端页面, 操作 -> 查看结果显示
- **后端**主要用于**处理数据**, 用户提交了数据 -> 处理好逻辑 -> 与数据库交互

**接口大体分类**

- 硬件接口 (USB口/网口/VGA视频接口/HDMI接口等)
- 软件接口 (软件测试只关注软件接口)

**概念**

有规范, 且用于通信的管道

**作用**

数据交互

**接口示例**

- 在百度搜索12306, 得到一个url: https://www.baidu.com/s?wd=12306
  - 哪一部分代表了通信的通道 ---> `/s`
  - 通信规则是什么? ---> `?代表路径和参数之间的间隔符　提交参数 wd=12306`
  - 接口完成了怎么样的数据交互?

**用python动手写一个接口**

代码块

```
# 如果需要作为服务来运行, 需要提前安装FastAPI, uvicorn
# pip install fastapi
```

```python
# pip install uvicorn

import uvicorn
from fastapi import FastAPI

app = FastAPI()

@app.get("/h")
def hello():
    return "hello world"

@app.get("/add")
def add(a: int, b: int):
    return a + b


uvicorn.run(app, host="localhost", port=8000)
```

> 接口的本质是什么?
>
> 一个能够被前端所访问的, 后端的函数

**面向接口开发示例**

1. 预定义接口文档, 称之为API文档 (Application Program Interface)
2. 前后端编码时, 参考API文档

> 举例: 开发登录接口
>
> - 编写登录接口的API文档
>
> 代码块
>
> ```
> 描述: 登录接口
> url: http://www.xxx.com/login
> 请求方式: POST
> 提交数据:
   账号 username
   密码 password
> 响应结果
   成功 True
   失败 False
> ```
>
> - 前端编码

> 代码块
>
> ```
> <form action="http://www.xxx.com/login" method="POST">
> 账号: <input type="text" name="username" />
> 密码: <input type="password" name="password" />
> <button type="submit">登录</button>
> </form>
> ```
>
> - 后端编码: 略, 获得登录用的数据 -> 查库并返回 True 或者 False 即可

**软件接口的分类**

- 按照客户端划分
  - web接口
  - app接口
  - 小程序接口
  - ...
- 按照协议划分
  - http接口
  - TCP接口
  - socket接口
  - webservice接口
  - ...
- 按开发人员归属划分
  - 自研接口
  - 第三方接口
- ...

**面试题**

什么是接口?[1]

- 具备一定规则的通信通道
- 本质上就是能够被前端所访问的后端的函数

## 1.2 接口测试介绍

**为什么要学习接口测试**

**场景**

某软件提供注册功能, 并且已经通过了功能测试, 但是项目上线一段时间后, 却发现被注册了大量垃圾用户, 比如: 有的账号或密码超长/为空 或 格式错误, 请问为什么?

...

要解决上述问题, 必须得学习接口测试

**概念**

绕过前端, 直接测试后端接口实现是否符合规范

**作用**

- 发现程序的安全隐患
- 提高效率
- **定位bug**

**面试题**

接口测试怎么定位bug?[2]

- 确认bug后, 对后端做接口测试, 如果没问题, 那么该bug是前端问题
- 如果后端有问题, 不能判断前端是否也存在问题, 可以在后端修复后, 再从界面上进行验证
  - 验证后如果bug消失了, 那么确认, 原来的问题就是后端引起的
  - 验证后如果bug还存在, 那么前端也有问题, 需要修复

**接口测试三要素**

- 定位接口资源 (通过路径找到接口)
- 提交测试数据
- 判断响应结果

**接口测试分类**

- 按测试目的进行划分
  - 接口功能测试
  - 接口回归测试

    - 也可以有接口冒烟测试...
- 按是否自动化运行划分
    - 接口(手工)测试
    - 接口自动化测试

## 2 restful 风格

### 2.1 概述

> 你可以这样定义用户的增删改查的接口路径
>
> - 查询 /getUser 新增 /addUser 修改 /updateUser 删除 /deleteUser
> - 查询 /findUser 新增 /newUser ....
>
> 开发人数众多, 会导致API文档可读性变差

**定义**

restful 是一种接口设计风格, 约束了接口实现的规范

> 如果使用 restful 风格来约束, 该如何定义用户的增删改查的接口路径
>
> - 路径统一使用 /user
> - 用请求方式区分操作, get 查询, post 新增, put 修改, delete 删除
> - 除此之外, restful 风格还约束了响应状态码和响应数据的格式等内容

**作用**

提高API文档的易读性

### 2.2 三要素

既然接口测试关注三要素, 那么API文档中对接口的描述必须交代清楚三要素

> 描述: 实现用户登录功能
> 三要素:
> 1.接口资源?
> 2.测试数据?
> 3.响应结果?

### 2.2.1 要素一 接口资源

**组成**

- URL (统一资源定位符), 如: http://127.0.0.1:8081/hello 或者 https://www.baidu.com/s
  - 协议: 常见的有 http/https 等
  - IP: 服务器IP地址
  - 端口: http默认80, https默认443
  - 路径: 通过路径, 可以找到对应的接口
- 请求方式 (数据的提交方式)
  - 常见实现有: get, post, put, delete, 分别对应 查, 增, 改, 删 四种操作

> 描述: 实现用户登录功能
> 三要素:
> 1.URL: http:www.xxx.com/login
> 请求方式: post
> 2.测试数据?
> 3.响应结果?

**面试题**

get和post比较, 有什么区别?[3]

- 提交方式不同, get提交的数据显示在地址栏, post是隐式的提交数据, 后者更安全
- 可提交的数据量不同, get提交的数据量有限制, 而post无限制
- 执行效率不同, get的执行效率比post稍高

### 2.2.2 要素二 测试数据

**两种常见的提交格式**

- 键值对格式
- JSON格式 (类似于 python 的字典)

**举例**

- 键值对

```
代码块
```

```
username=jiege&password=123&aihao=chuiniu&aihao=xuexi
```

- JSON

代码块

```
{
  "username":"jiege",
  "password":"123",
  "aihao":["chuiniu", "xuexi"]
}
```

> 描述: 实现用户登录功能
> 三要素:
> 1.URL: http:www.xxx.com/login
> 请求方式: post
> 2.键值对格式: username=xxx&password=yyy
> 3.响应结果?

### 2.2.3 要素三 响应结果

**两大关注点**

- 状态码
  - 1xx: 请求正常, 但是无响应, 只在实验状态下使用
  - 2xx: 请求正常, 响应正常, 比如: 200查询成功 201新增或修改成功 204删除成功
  - 3xx: 以其他方式获取响应, 比如: 302重定向 304取本地缓存
  - 4xx: 浏览器端异常, 比如: 401未授权 403拒绝访问 404资源路径有误
  - 5xx: 服务器端异常, 比如: 500服务器运行异常 502网关异常 504网关超时
- 响应体
  - 最常见的响应体是JSON格式的数据, 如: {"code": 0, "data": [], "msg": "登录成功"}

> 描述: 实现用户登录功能
> 三要素:
> 1.URL: http:www.xxx.com/login
> 请求方式: post

> 2.键值对格式: username=xxx&password=yyy
> 3.状态码: 200 响应体: {"code": 0, "data": [], "msg": "登录成功"}
> 状态码: 200 响应体: {"code": 1001, "data": [], "msg": "密码错误, 登录失败"}
> 状态码: 200 响应体: {"code": 1002, "data": [], "msg": "账号被锁定, 登录失败"}

**面试题**

常见的状态码有哪些?[3]

- 200一般是代表查询成功, 201代表新增或修改成功, 204代表删除成功
- 302代表重定向, 304取本地缓存
- 401代表未授权, 403代表拒绝访问, 404代表路径错误
- 500代表服务器异常, 504网关超时

### 2.2.4 小结

接口虽多, 但是不外乎增删改查四种类型, 且每种接口类型的实现也都由三要素组成

| 请求类型 | 要素一 | 要素二 | 要素三 |
| --- | --- | --- | --- |
| 查 | URL + get | 键值对 | 状态码: 200 响应体: 单条或多条数据 |
| 增 | URL + post | 键值对 / JSON | 状态码: 201 响应体: 新增后的数据 |
| 改 | URL + put | 键值对 / JSON | 状态码: 201 响应体: 修改之后的数据 |
| 删 | URL + delete | 键值对 | 状态码: 204 响应体: 删除后的数据 |

> restful 只是规则, 不是标准, 换言之, 并不是硬性约束

**三要素和行头体的对应关系**

| | | | |
| --- | --- | --- | --- |
| **请求行** | **URL, 请求方式** | **响应行** | **http协议版本, 状态码** |
| **请求头** | **支持语言, 浏览器信息** | **响应头** | **web服务器信息, 响应内容的类型, 响应内容的长度** |
| **请求体** | **请求参数** | **响应体** | **http响应的具体数据** |

**面试题**

http接口有哪些类型?[2]

- 操作的类型主要增删改查这四种
- 查询类型一般对应get这种请求方式

- 新增用post, 修改用put, 删除用delete

请求头和响应头中有哪些常见的字段, 以及分别是什么作用?[2]

- 请求头
  - Accept 表示浏览器可以接受的响应数据的类型
    - Accept-Encoding 用于指定压缩方法, Accept-Language 申明接收的语言
  - Content-Type 客户端告诉服务器, 我们发送的数据类型, 方便服务器去解析数据
  - User-Agent 告诉服务器, 客户端的操作系统和浏览器名称及版本
  - Cookie 用来存储一些用户信息, 以便让服务器辨别用户身份
- 响应头
  - Content-Length 响应消息的长度
  - Server 服务器信息(如 Apache/1.3.27 Red-Hat/Linux)
  - Set-Cookie 告诉客户端, 把什么信息设置进Cookie

http和https的区别?[3]

- 默认端口不同, 前者是80, 后者是443
- 安全性不同, 前者是明文传输, 后者是ssl加密的, 后者更安全
- http不需要证书, https协议需要到CA申请证书, 一般需要一定的费用

## 2.3 案例

判断如下API文档是否符合restful规范

代码块

```
一.查询
    1.1 学员-查询所有
        请求方法: GET
        请求地址: http://127.0.0.1:8000/api/students
    1.2 学员-查询指定单个
        请求方法: GET
        请求地址: http://127.0.0.1:8000/api/students?s_id=S001
        (注: s_id 为参数名称; S001,S005,S088 为学员ID;)
    1.3 学员-查询指定多个
        请求方法: GET
        请求地址: http://127.0.0.1:8000/api/students?s_id=S001,S005,S088
                    (注: s_id 为参数名称; S001,S005,S088 为学员ID;)
```

```
    1.4 学员-组合查询
        请求方法: GET
        请求地址: http://127.0.0.1:8000/api/students?chengji=A&banji=B01
        (注: chengji: 成绩; banji: 班级;)

二.新增
    2.1 学员-新增
        1) 请求方法: POST
        2) 请求地址: http://127.0.0.1:8000/api/newstudents/
        3) 请求JSON报文:
        4) 调用传入的json串如下（可新增多条，之间用,隔开）:
            {
                "data": [
                    {
                        "s_id":"S009",
                        "s_name":"zhangsanfeng",
                        "banji":"B01",
                        "chengji":"A"
                    }
                ]
            }
            5) 新增成功返回报文:
            {
                "already_exist": {
                    "results": [],
                    "count": 0
                    },
                 "create_success": {
                    "results": [
                        {
                            "s_id":"S009",
                            "s_name":"zhangsanfeng",
                            "banji":"B01",
                            "chengji":"A"
                        }
                    ],
                    "count": 1
                }
            }
            6) 新增失败id已存在-返回报文:
            {
                "already_exist": {
                    "results": [
                        {
                            "s_id":"S009",
                            "s_name":"zhangsan",
                            "banji":"B02",
```

```
                            "chengji":"B"
                        }
                    ],
                    "count": 1
                },
                "create_success": {
                    "results": [],
                    "count": 0
                }
            }
            7) 新增失败json格式错误:
            {
                "status_code": 400,
                "detail": "请求体参数格式错误。"
            }

三.更新
    3.1 学员-更新
        1). 请求方法: PUT
        2). 请求地址: http://127.0.0.1:8000/api/updatestudents/S009/
                        (注: s009为学员ID)
        3). 请求JSON报文:
            {
                "data": [
                    {
                        "s_id":"S009",
                        "s_name":"zhangsanfeng",
                        "banji":"B01",
                        "chengji":"A"
                    }
                ]
            }
        4). 修改成功返回:
            {
                "s_id":"S009",
                "s_name":"zhangsanfeng",
                "banji":"B01",
                "chengji":"A"
            }
四.删除
    4.1 学员-删除单个
        请求方法: DELETE
        请求地址: http://127.0.0.1:8000/api/deletestudents/S003/
        (注:S003为学员ID)
```

## 3 接口测试环境

🔖 **背景:**

- **学习视频** `07 接口测试-接口测试基础-3-接口测试环境` **中会提到api1.0的接口代码实现原理/api1.0.jar在本机的运行依赖和启动过程/对应的数据库的创建过程等, 较为繁琐, 以后统一使用虚拟机1中的api1.0即可**

**目标: 轻松拥有属于自己的api1.0项目**

**内容: 使用虚拟机1**

- 虚拟机1的应用4: api1.0项目的启动和访问

**环境分析**

1. 项目 (后端), 案例由Java编写, 需要编译后才能运行, 依赖的内容有:
   - JDK
   - IDE (开发需要的)
   - Tomcat (这里我们直接给出编译打包后的文件, 内置了tomcat)
   - MySQL
2. 测试工具, 如:
   - 浏览器
   - postman
   - Jmeter

**项目部署和运行**

- 后端
  - 在jar文件所在目录, 命令行运行 `java -jar api1.0.jar` 即可
- 数据库
  - 创建一个名为api_test的数据库 (root/123456)
  - 运行api_test.sql

**测试工具**

- 浏览器, 本身只能帮助测试get方式的请求 (不算插件)
- postman: 主流工具之一, 功能强大
- jmeter: 开源, 支持多种协议, 免安装, 可做性能测试

> 官网地址: https://jmeter.apache.org/, 选择Binaries版本进行下载即可

**项目API文档**

后续用到时, 备查

```
代码块
一.查询
    1.1 查询所有区域信息
        请求方法: GET
        请求地址: http://127.0.0.1:1234/area/listarea
    1.2 查询指定单个区域信息
        请求方法: GET
        请求地址: http://127.0.0.1:1234/area/getareabyid?areaId=1
        (注: areaId为参数名称; 1 为区域ID;)

二.新增
    2.1 新增区域信息
        1) 请求方法: POST
        2) 请求地址: http://127.0.0.1:1234/area/addarea
        3) 请求键值对: areaName="xxx"&priority="yyy"
            5) 新增成功返回报文:
            {success: true}


三.更新
    3.1 更新区域信息
        1). 请求方法: PUT
        2). 请求地址: http://127.0.0.1:1234/area/modifyarea
        3). 请求json报文:
            {
                                "areaId": 2,
                                            "areaName": "xxx",
                                            "priority": "yyy"
            }
        4). 修改成功返回:
            {success: true}
四.删除
    4.1 删除单个区域信息
        请求方法: DELETE
        请求地址: http://127.0.0.1:1234/area/removearea?areaId=2
```

# 4 Jmeter基础

## 4.1 基本使用流程

**需求**

使用 jmeter 访问项目案例的查询接口

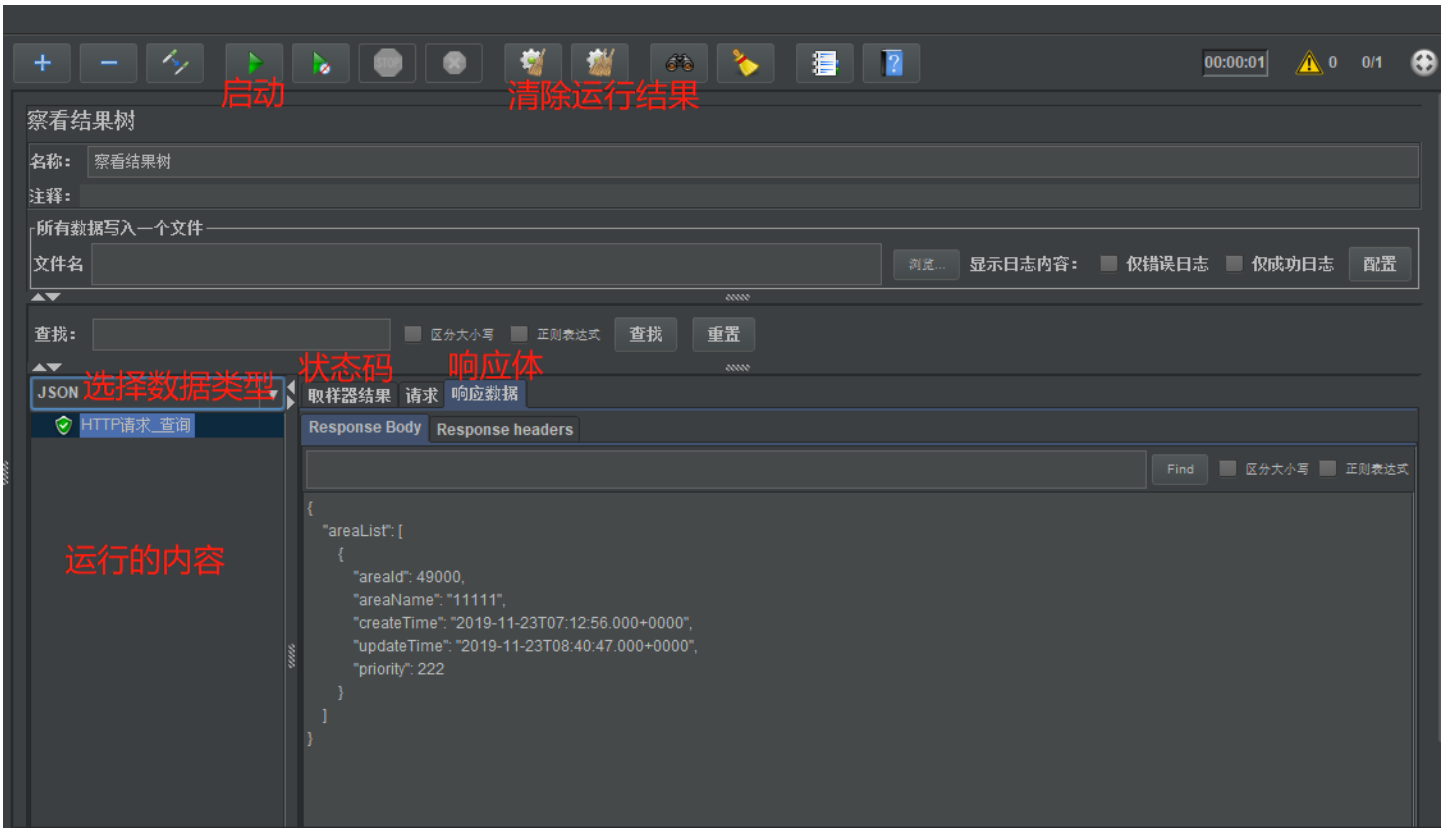
**使用步骤**

1. 启动 jmeter 并搭建基本框架, 添加线程组 + HTTP请求 + 查看结果树

> 养成 ctrl + s 保存脚本的习惯

2. 设置接口测试三要素

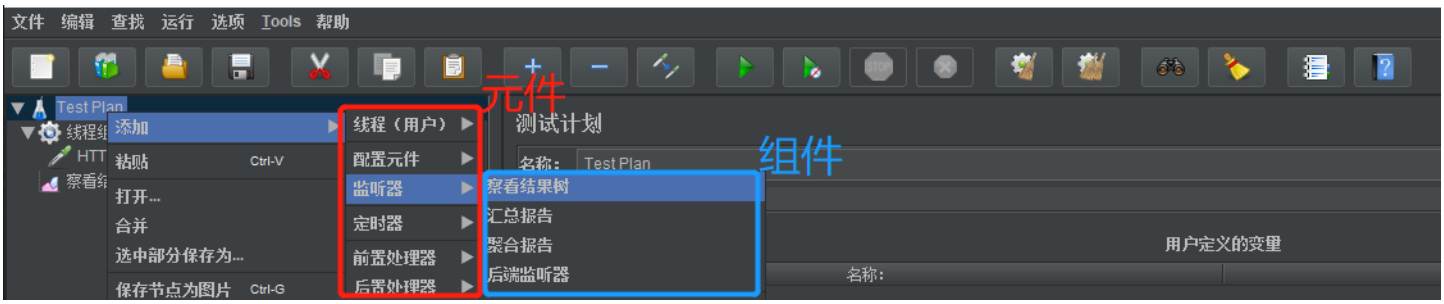
3. 运行并查看结果




**组件与元件**

组件: 是 Jmeter 中的一些功能点实现(比如: 线程组、http请求、查看结果树 .....)

元件: 对组件按照性质归类分组，作用:方便管理组件




## 4.2 组件-线程组

### 4.2.1 总体介绍

**概念**

- 进程: 正在运行的程序
- 程序启动，进程创建，程序关闭，进程释放，反之，进程释放，程序关闭
- 线程: 程序中的执行线索(路径)
- 迅雷中每一部电影下载，游戏中的每一个角色
- 线程组: 对线程分类归组

> 层级关系:
>
> 进程 > 线程组 > 线程 === 一个进程可以包含多个线程组，一个线程组可以包含多个线程

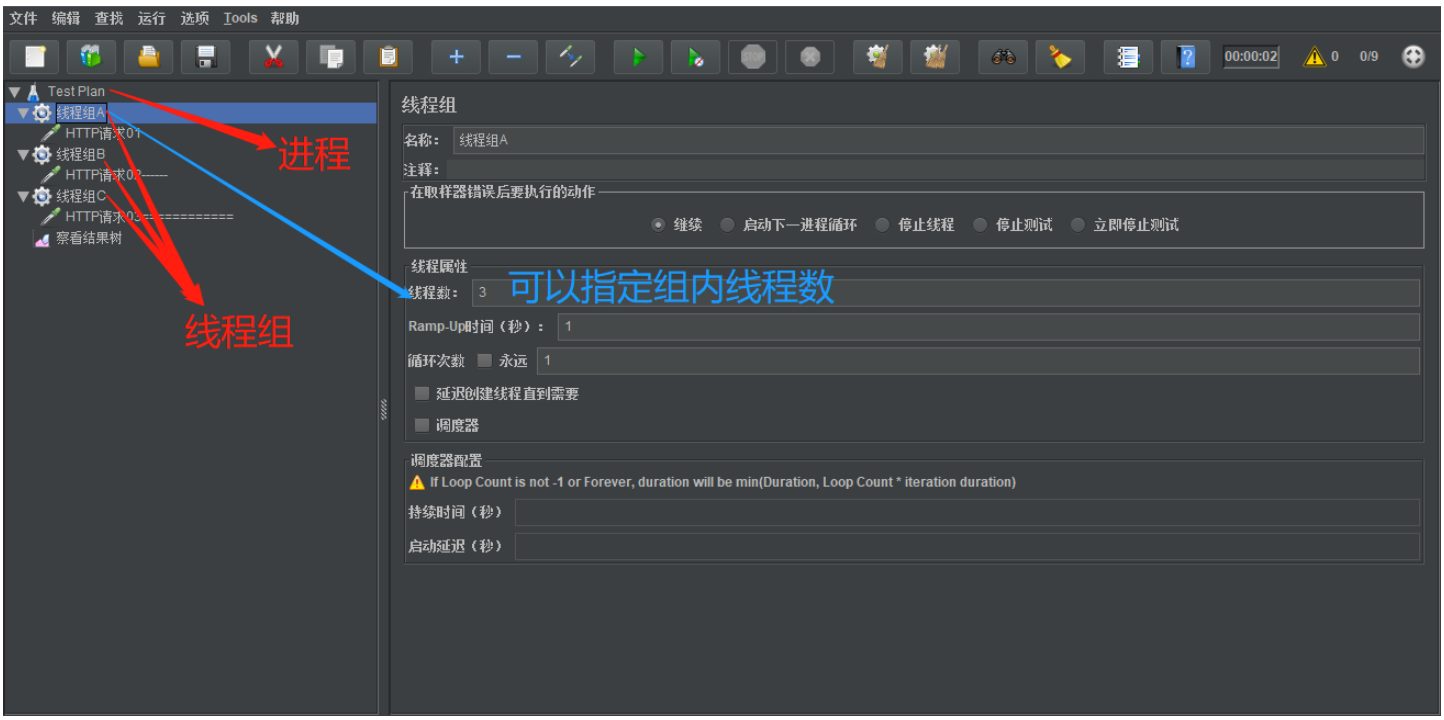
代码块

```
1   并发执行：多个线程同时执行          ===  多部电影同时下载   ====  线程的启动顺序和结束顺序不一定一致
2   顺序执行：多个线程按照顺序依次执行   ===  电影下载后播放     ====  线程的启动顺序和结束顺序一定一致
```

**作用**

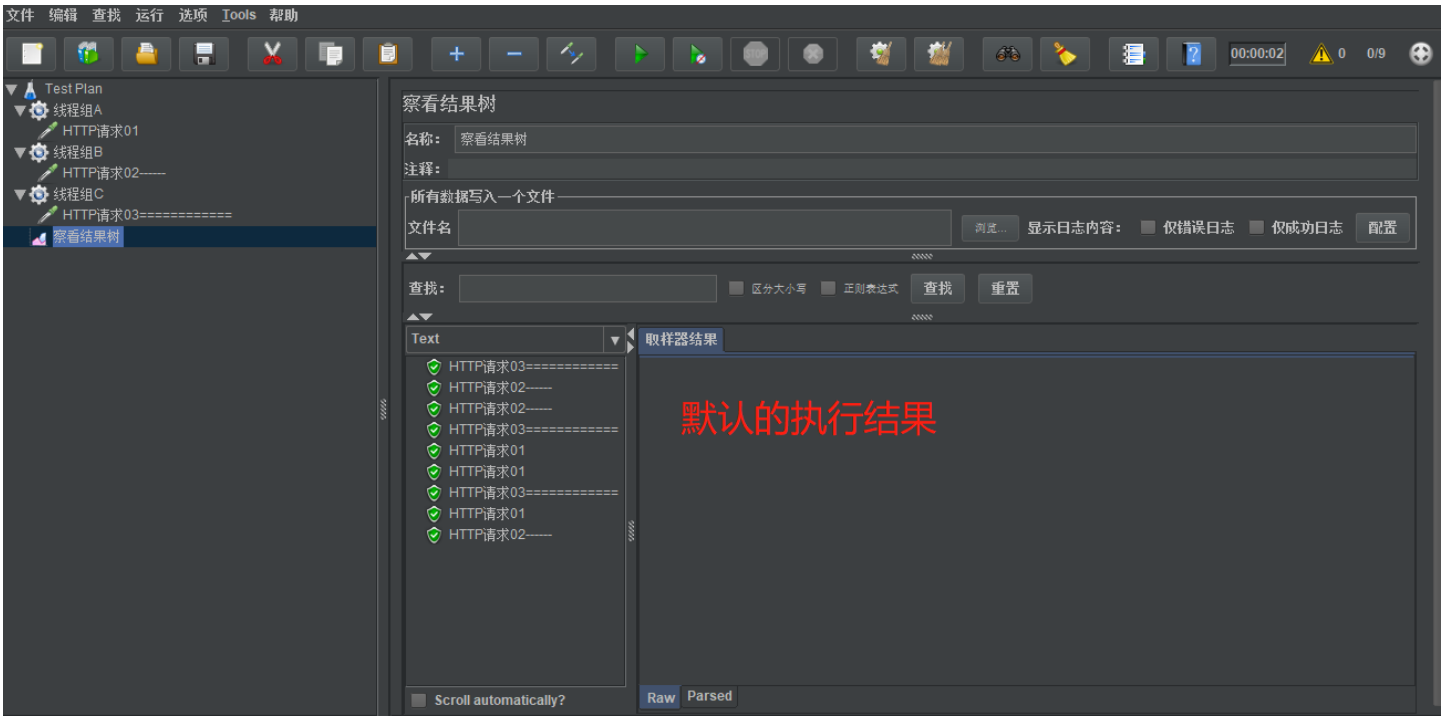
方便管理线程

### 4.2.2 准备层级关系

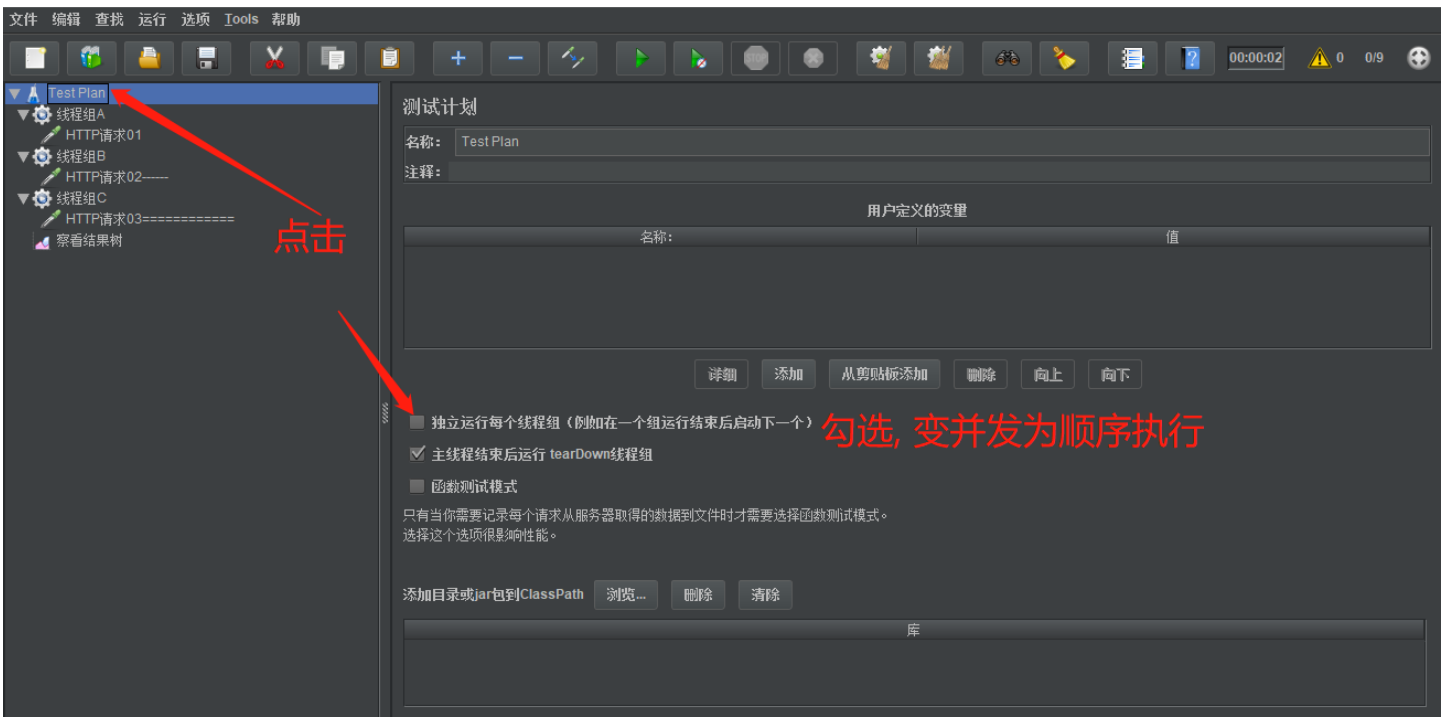



### 4.2.3 执行顺序

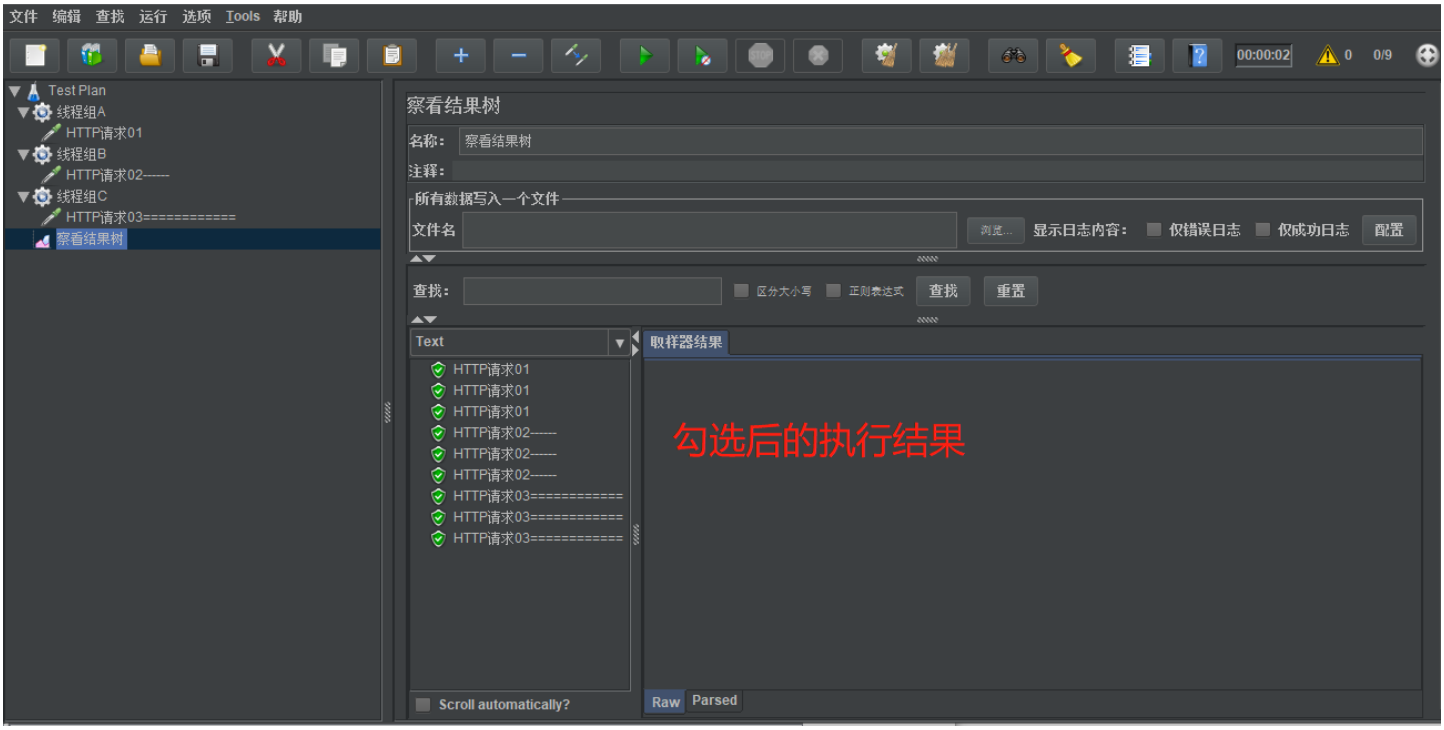
**修改前默认**


文件 编辑 查找 运行 选项 Tools 帮助
00:00:02
0 0/9
Test Plan
线程组A
HTTP请求01
线程组B
HTTP请求02------
线程组C
HTTP请求03==========
察看结果树
察看结果树
名称：
注释：
所有数据写入一个文件
文件名
浏览...
显示日志内容：
仅错误日志
仅成功日志
配置
查找：
区分大小写
正则表达式
查找
重置
Text
取样器结果
HTTP请求03==========
HTTP请求02------
HTTP请求02------
HTTP请求03==========
HTTP请求01
HTTP请求01
HTTP请求03==========
HTTP请求01
HTTP请求02------
默认的执行结果
Scroll automatically?
Raw
Parsed

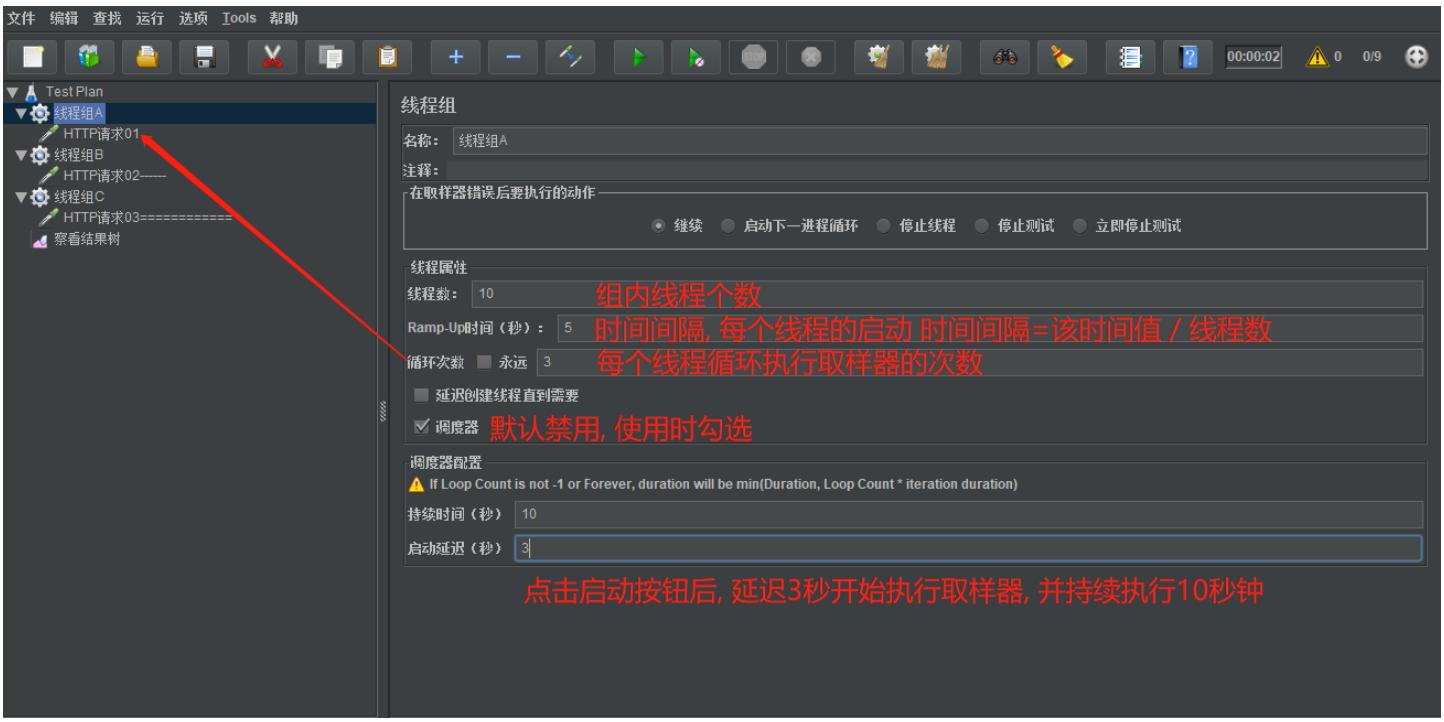
## 修改


文件 编辑 查找 运行 选项 Tools 帮助
00:00:02
0 0/9
Test Plan
线程组A
HTTP请求01
线程组B
HTTP请求02------
线程组C
HTTP请求03==========
察看结果树
点击
测试计划
名称：
Test Plan
注释：
用户定义的变量
名称：
值
详细
添加
从剪贴板添加
删除
向上
向下
独立运行每个线程组（例如在一个组运行结束后启动下一个）
勾选，变并发为顺序执行
主线程结束后运行 tearDown线程组
函数测试模式
只有当你需要记录每个请求从服务器取得的数据到文件时才需要选择函数测试模式。
选择这个选项很影响性能。
添加目录或jar包到ClassPath
浏览...
删除
清除
库

## 修改后




### 4.2.4 常见属性设置




> 线程数设置为 3 循环次数设置为1， 与线程数设置为1 循环次数设置为 3，执行结果一致，有区别吗?
>
> 有区别，前者模拟的是 3 个用户每个用户访问1次服务器资源，后者是1个用户循环访问3次服务器

**面试题**

线程组中的线程数, 启动时间, 循环次数分别代表什么意思?[1]

- 线程数, 组内线程个数, 相当于用户数
- 启动时间, Ramp-Up时间, 代表在多少秒内完成所有线程的启动, 一般默认即可
- 循环次数, 代表每个线程循环执行取样器的次数

### 4.2.5 特殊线程组

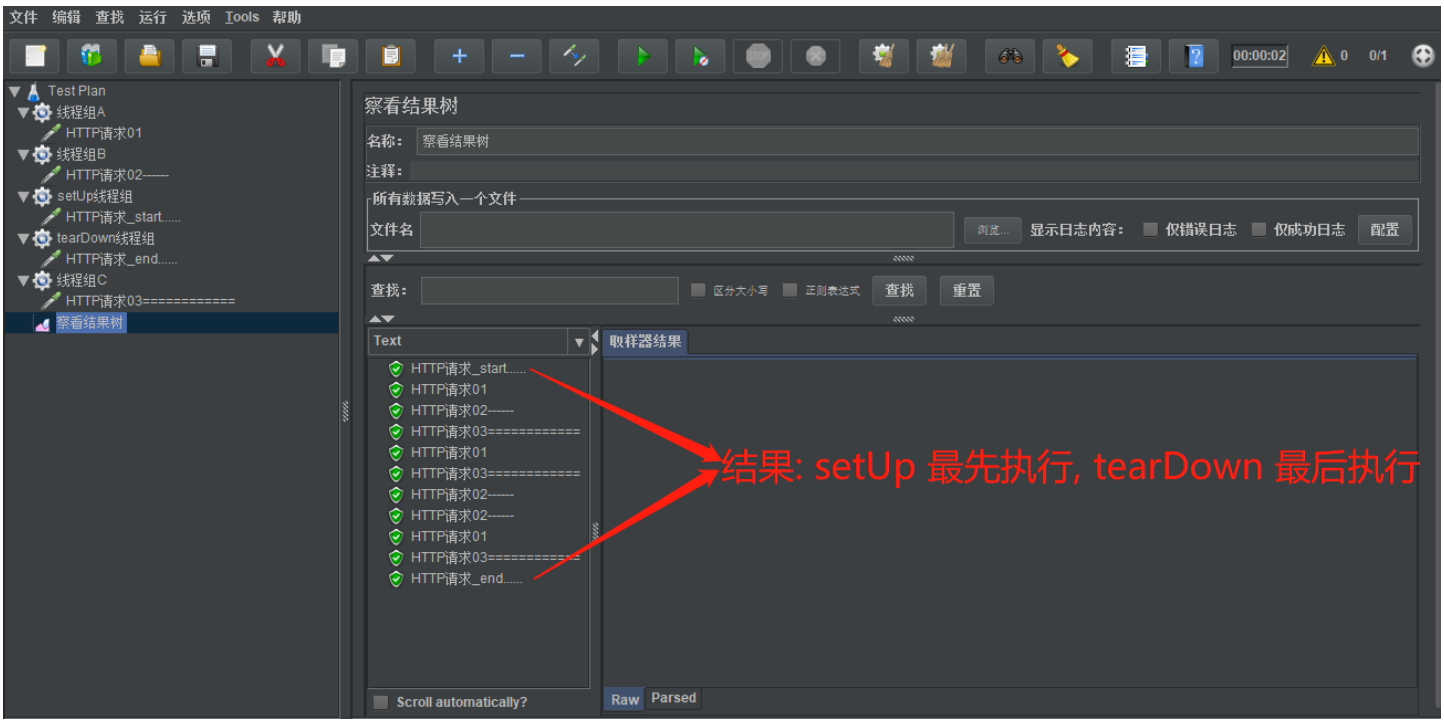
**setUp 线程组:** 最先执行的线程组，一般用于初始化操作

**tearDown 线程组:** 最后执行的线程组，一般用于资源销毁

**举例**

添加: 右键 测试计划 -> 添加 -> 线程 (用户) -> 可以找到特殊线程组

执行结果:




### 4.2.6 查看结果树的作用域

查看结果树显示线程组内取样器的执行结果的，查看结果树添加位置不同，对应生效的取样器也不同

- 添加在测试计划下: 对所有线程组的所有取样器生效
- 添加在线程组下: 只对当前组内取样器生效
- 添加在取样器下: 只对当前取样器生效

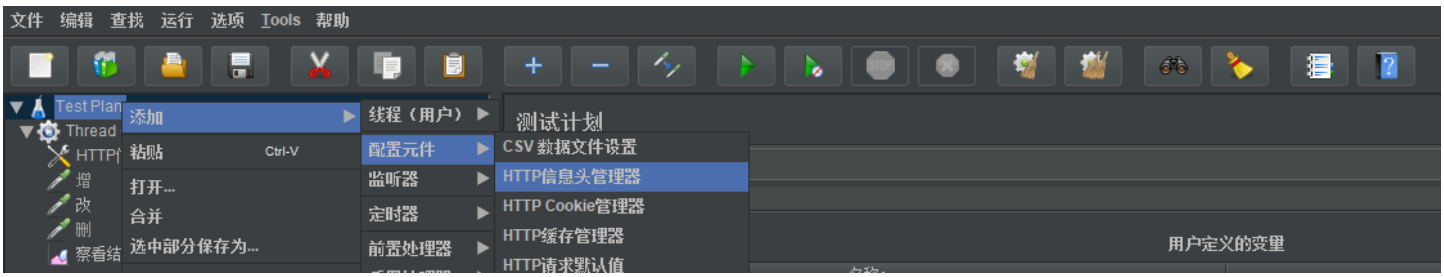
## 4.3 练习

**需求**

使用 jmeter 访问案例的 增/ 改/ 删 接口

### 4.3.1 JSON格式参数

提交的数据是 JSON 格式时，需声明提交的数据的内容类型




### 4.3.2 http请求默认值

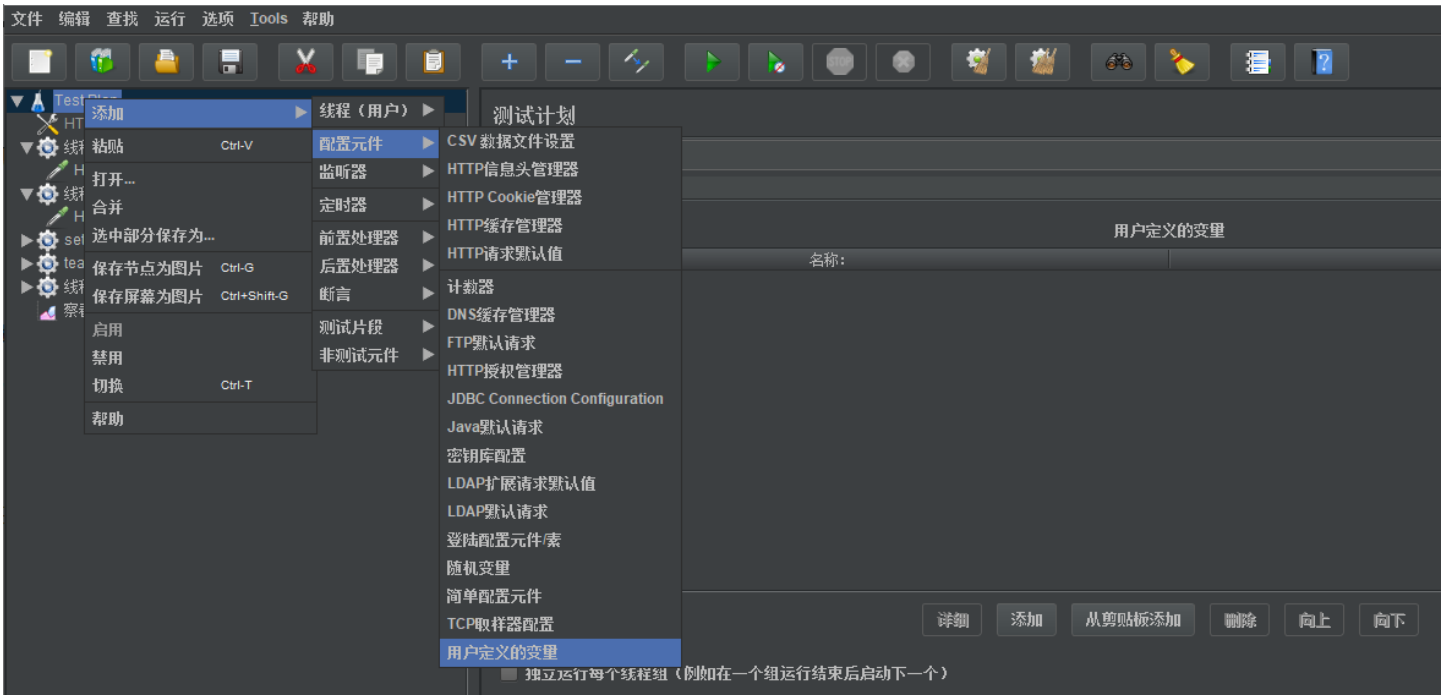
随着调试接口的数量增加, 发现多数http请求中**IP**, **端口**, **编码**基本相同, 可以使用**HTTP请求默认值**简化脚本

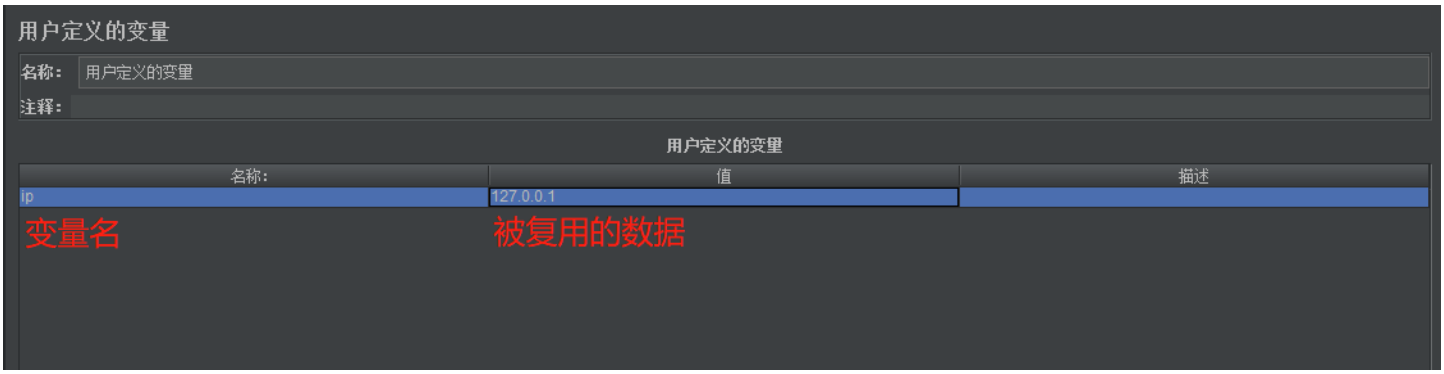
### 4.3.3 用户定义的变量

不同环境的 ip/端口 是不同的, 当需要修改环境的时候, 逐个修改每个请求的 ip/端口 是低效的, 即便使用HTTP请求默认值, 修改起来也稍有繁琐, 可以使用**用户定义的变量**来简化操作

> 使用步骤:
>
> 1. 定义变量
> 2. 引用变量

**定义变量**




### 引用变量

使用 `${变量名}` 的方式对变量进行引用

### 使用场景

切换不同测试环境时, 可以以如下方式来使用:

1. 分别使用两个组件**用户定义的变量**, 来定义两个环境的变量
2. 在**HTTP请求默认值**中引用变量
3. 通过控制**用户定义的变量**组件的启用/禁用, 来控制请求的是哪个环境的接口

## 5 测试流程扩展

### 没有接口测试时的测试流程

```
代码块
```

```
1.需求分析,需求评审
2.编写测试计划
3.编写测试用例以及发起用例评审
4.[可选]搭建测试环境
5.系统测试,编写测试报告
        a.冒烟测试
        b.第一轮--新功能测试
        c.第二轮--bug回归
        d.第三轮--bug回归
        e....
        f.最后一轮旧功能回归
6.验收测试,编写测试报告
7.线上,点点点
```

**思考**

接口测试应该在什么环节引入?

引入接口测试用来解决什么问题?

**引入接口测试后的测试流程**

代码块

```
1.需求分析,需求评审
2.编写测试计划
3.编写测试用例以及发起用例评审   --> 可以协助开发联调接口(通过性测试), 接口功能测试
4.[可选]搭建测试环境
5.系统测试,编写测试报告
        a.冒烟测试
        b.第一轮--新功能测试
        c.第二轮--bug回归
        d.第三轮--bug回归
        e....
        f.最后一轮旧功能回归   -->可以用接口回归测试来替代
6.验收测试,编写测试报告
7.线上,点点点
```

# 单接口测试

**应用场景**

- 前后端编码/联调阶段, 后端接口相继写好, 测试人员可以协助联调接口
  - 特点1: 难度较高, 需要自己造数据
  - 特点2: 测试目的是看接口通不通, 交互的数据是否正确

> 代码块
>
> ```
> #联调时的常规对话:
>
> 后端：接口好了，你试试
>
> 前端：不行，500
>
> 后端：我看看
>
> ~~~半个小时飘过~~~
>
> 后端：好了，修复了个bug，你再试试
>
> 前端：不行呀，还是500
>
> ~~~又过了很久……~~~
>
> 后端：好了，参数改了，接口文档有更新，你看看
>
> 前端：好的（心里MMP）
>
> 测试：周四一早提测哦
>
> ~~~前后端心中响起，凉凉……~~~
> ```

- 前后端编码/联调阶段, 测试人员设计各种正向+逆向的用例, 对接口功能进行验证

# 6 接口功能测试

## 6.1 概述

**含义**

模拟用户的多样性操作，查看实际结果是否符合预期

**原则**

- 设计各种正向 + 逆向的测试用例

- 接口覆盖率达到100%

## 6.2 测试流程

1. 根据API文档, 编写测试计划
2. 根据API文档, 设计接口功能测试用例, 并发起评审
3. 使用工具 (如, Jmeter) 编写测试脚本, 执行测试用例
4. 输出测试报告

### 6.2.1 编写测试计划

编写测试计划的核心目的是**分配资源**, 如:

| 任务名称 | 任务描述 | 责任人 | 工期 | 开始日期 | 结束日期 | 进度 | 备注 |
|---|---|---|---|---|---|---|---|
| 环境搭建 | 安装并运行学生管理系统 | 运维组葫芦娃 | 1 | 1月1日 | 1月2日 | 进行中 | |
| 学院模块接口测试 | 测试学院增删改查接口 | 测试一组凹凸曼 | 3 | 1月2日 | 1月5日 | 未开始 | |
| 班级模块接口测试 | 测试班级增删改查接口 | 测试一组穿山甲 | 3 | 1月2日 | 1月5日 | 未开始 | |
| 学生模块接口测试 | 测试学生增删改查接口 | 测试二组哥斯拉 | 4 | 1月2日 | 1月6日 | 未开始 | . |

### 6.2.2 设计测试用例

**新增学院接口**

- 请求路径：/api/academy
- 请求方法：post
- 请求参数:

| 参数名 | 参数说明 | 备注 |
|---|---|---|
| academyName | 学院名称 | 汉字+字母+数字, 10个字符以内, 不能为空, 唯一 |
| describe | 学院描述 | 汉字+字母+数字, 10个字符以内, 可以为空 |

- 响应参数：

| 参数名 | 参数说明 | 备注 |
|---|---|---|
| id | 学院id | 自增且唯一 |

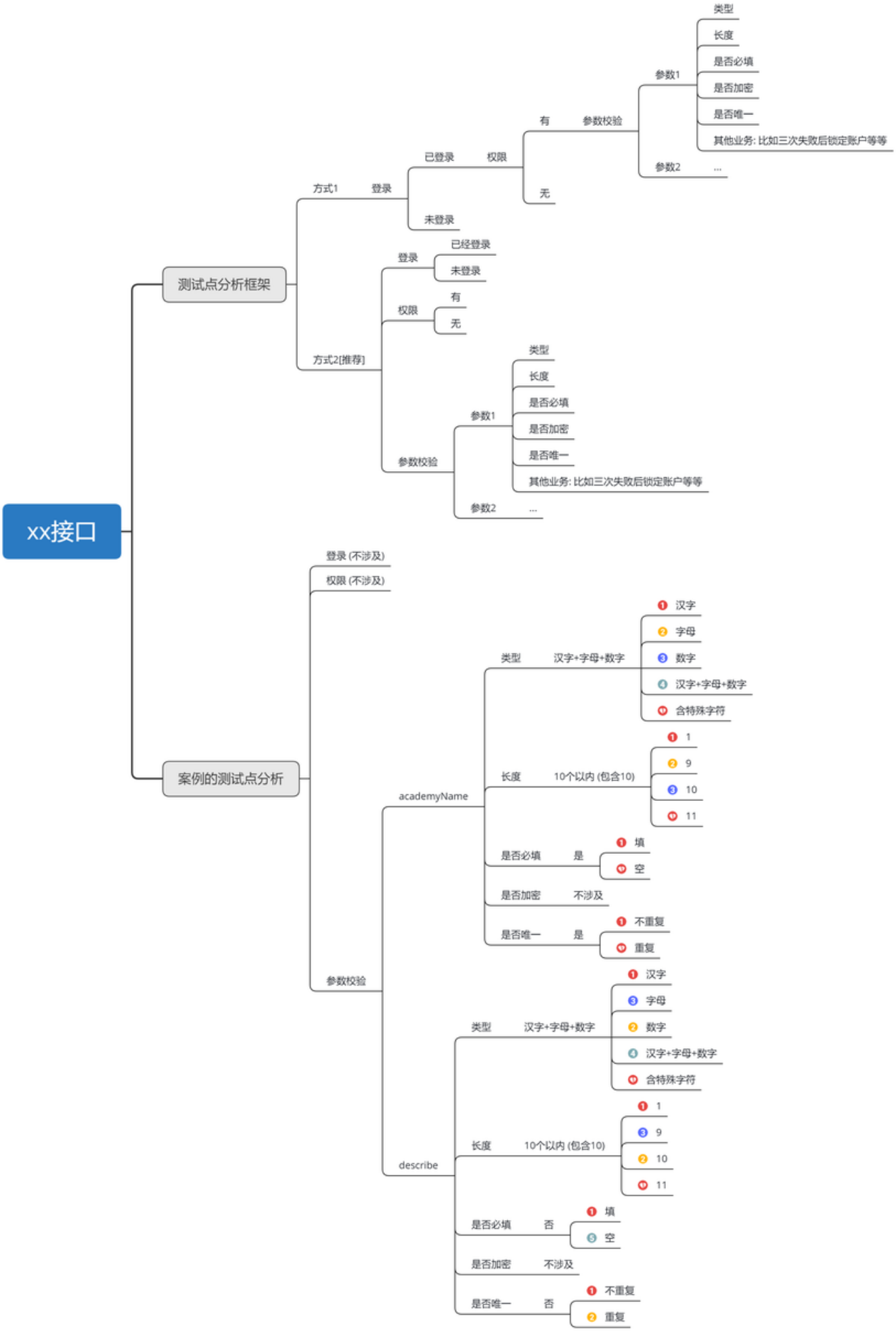
| | | |
|---|---|---|
| academyName | 学院名称 | |
| describe | 学院描述 | |

- 响应数据示例

代码块

```
新增学院成功:
{
        "code": 0,
        "data": [{"id": 1001,
                  "academyName": "计算机学院",
                  "describe": ""}]
        "msg": "新增学院成功"
}
新增学院失败（学院名称含有特殊字符）:
{
        "code": 1001,
        "data": []
        "msg": "错误，学院名称类型错误"
}
新增学院失败（学院名为空）:
{
        "code": 1002,
        "data": []
        "msg": "错误，请填写学院名"
}
...
```

### 分析测试点


xx接口
测试点分析框架
方式1
登录
已登录
权限
有
参数校验
参数1
类型
长度
是否必填
是否加密
是否唯一
其他业务: 比如三次失败后锁定账户等等
参数2
...
无
未登录
方式2[推荐]
登录
已经登录
未登录
权限
有
无
参数校验
参数1
类型
长度
是否必填
是否加密
是否唯一
其他业务: 比如三次失败后锁定账户等等
参数2
...
案例的测试点分析
登录 (不涉及)
权限 (不涉及)
参数校验
academyName
类型
汉字+字母+数字
1 汉字
2 字母
3 数字
4 汉字+字母+数字
9 含特殊字符
长度
10个以内 (包含10)
1 1
2 9
3 10
9 11
是否必填
是
1 填
9 空
是否加密
不涉及
是否唯一
是
1 不重复
9 重复
describe
类型
汉字+字母+数字
1 汉字
3 字母
2 数字
4 汉字+字母+数字
9 含特殊字符
长度
10个以内 (包含10)
1 1
3 9
2 10
9 11
是否必填
否
1 填
5 空
是否加密
不涉及
是否唯一
否
1 不重复
2 重复

**测试用例**

详见 `接口用例模板.xlsx`

**面试题**

如何写接口测试用例?[3]一个接口一般写多少条测试用例?[1]

- 单个参数类型, 长度
  - 各种类型 (字符串, 数字, 特殊字符等)
  - 各种长度, 注意长度边界和超长问题
- 必填项
  - 必填参数全传, 看是否返回正确结果
  - 单个必填项不传, 看是否请求失败
- 参数组合
  - 正向: 一条测试用例尽量覆盖更多的测试点
  - 逆向: 逆向的测试点, 每个点都是一条测试用例
- 接口安全
  - 加密问题, 敏感数据是否有加密, 加密规则是否复杂
  - 权限问题, 绕过身份认证, 是否能访问接口, 没有授权的情况, 能不能访问接口

> 一个接口对应多少条测试用例?
>
> - 一般一个参数对应的逆向的测试点大概有3.4个, 多个参数 * n
> - 实际工作中, 如果非要写测试用例的话, 一般一个有4.5个参数的接口, 测试用例大概15~20条
> - (实际工作中, 时间比较紧张, 接口的功能测试一般是不需要写测试用例的)

### 6.2.3 编写测试脚本

**实现方式**

> 核心知识
>
> - csv文件, 用于存储数据
>  - csv 文件其实就是格式化的txt文件, 直接改后缀可以相互转换
>  - csv 用英文逗号分隔值, 是程序中常见的数据存储格式
>  - csv 文件中一行代表一条数据, 一列代表一个字段

> - 核心组件: **CSV数据文件设置**, 用于读取数据

- 准备一个csv文件, 把数据写入, 参考 `接口功能测试数据.csv`

接口功能测试数据.csv

```
ID,参数1_academyName,参数2_describe,标题
academy_001,计算机@学院,,新增一个学院-学院名称含特殊字符-新增失败
academy_002,,123,新增一个学院-学院名称为空-新增失败
academy_003,11位的计算机学院名称,666,新增一个学院-学院名称超长-新增失败
```

- 读取数据, 添加**CSV数据文件设置**, 并配置

- 引用数据

**面试题**

jmeter中如何使用csv, 有什么实际的应用场景?[1]

- jmeter中使用csv数据文件比较简单, 有个组件叫 `csv数据文件设置` , 可以引入csv并使用csv内的数据
- 一般使用csv数据文件是在需要做接口功能测试时, 因为测试用例较多, 我们把测试数据提前放到csv文件中

### 6.2.4 输出测试报告

`接口用例模板.xlsx` 补充了测试结果, 测试人 , 测试时间, 备注等字段后, 可以视为**接口功能验证表**

## 7 参数化小结

### 7.1 概述

**目前已经用到的参数化方式**

- 用户定义的变量
  - 提前定义变量, 需要时引用, 修改方便, 常用于环境参数配置
- csv数据文件设置
  - 接口功能测试, 多条测试用例, 动态读取数据

**参数化的概念**

将静态数据转换为动态数据的过程, 用于动态的生成, 设置或导入数据

**作用**

- 高效
- 提高脚本的编写质量, 可维护性更好

## 7.2 实现方式介绍

**Jmeter中常用的参数化实现方式**

- 用户定义的变量
- csv数据文件设置
- 函数

> 三者互补, 重点是csv数据文件设置

## 7.3 函数

**需求**

访问查询area接口20次, 在结果树为请求添加标号

**步骤**

1. 编写脚本
2. 打开 jmeter 内置的 `函数助手`
3. 选择所需函数 (counter), 输入参数后, 点击 `生成`
4. 调用, 复制粘贴生成后的结果即可

> 记得把线程组的循环次数设置为20

**常用函数介绍**

- counter() -> 计数器, 参数1: TRUE | FALSE
  - TRUE, 一个线程对应一个单独的计数器
  - FALSE, 所有线程共用一个计数器
- random() -> 随机数, 参数1: 最小值, 参数2: 最大值
- time() -> 时间函数, 参数1: 格式化时间 yyyy/MM/dd HH:mm:ss 相当于年/月/日 时:分:秒

## 8 断言

**概念**

程序代替人工, 判断响应结果是否符合预期

**作用**

- 高效
- 安全

**常用断言方式**

响应断言, 判断响应数据中包含预期结果

- 如果包含, 断言成功 (说明本条测试用例执行是通过的)
- 如果不包含, 断言失败 (说明本条测试用例执行是不通过的)

**实现步骤**

1. 准备脚本
2. 添加断言 (响应断言, 同时添加断言包含的字符串内容)
3. 运行并查看结果
   - 通过, 没有提示
   - 不通过, 有错误提示

**断言应用示例**

1. 准备脚本 (使用新增一个学院信息的测试脚本)

2. 在 `接口功能测试数据_优化_增加断言.csv` 中增加一列数据, 数据内容是需要断言的内容
3. 在jmeter脚本中引用数据文件, 并在响应断言中动态的引用断言的内容

**常用断言方式[拓展]**

JSON断言, 判断响应数据中是否包含预期结果

> **注意**
>
> - JSON表达式中, `$` 是根节点, `.` 表示子节点
>  - 如果遇到数组类型的数据, 用 `[下标]` 来进行选择, 下标从0开始, `*` 是通配符
>  - 学习地址: https://goessner.net/articles/JsonPath/
> - Additionally assert value 这个选项需要勾选, 才能对值作断言, 默认不勾选, 只断言路径是否存在

## 9 连接数据库

**原理**

jmeter连接数据: jmeter -> 驱动 -> 数据库

**应用场景**

- 比如测试新增一个area信息的接口, 可以去数据库查看该数据是否新增成功, 用于断言
- 比如执行测试用例后, 需要清除垃圾数据, 就需要连接数据库, 并执行删除表数据的操作, 用于销毁测试数据

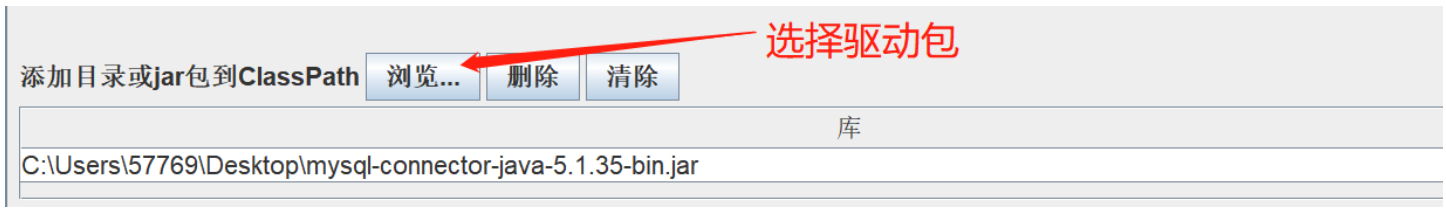
**需求案例**

新增一个area信息, 查询数据库并断言新增成功

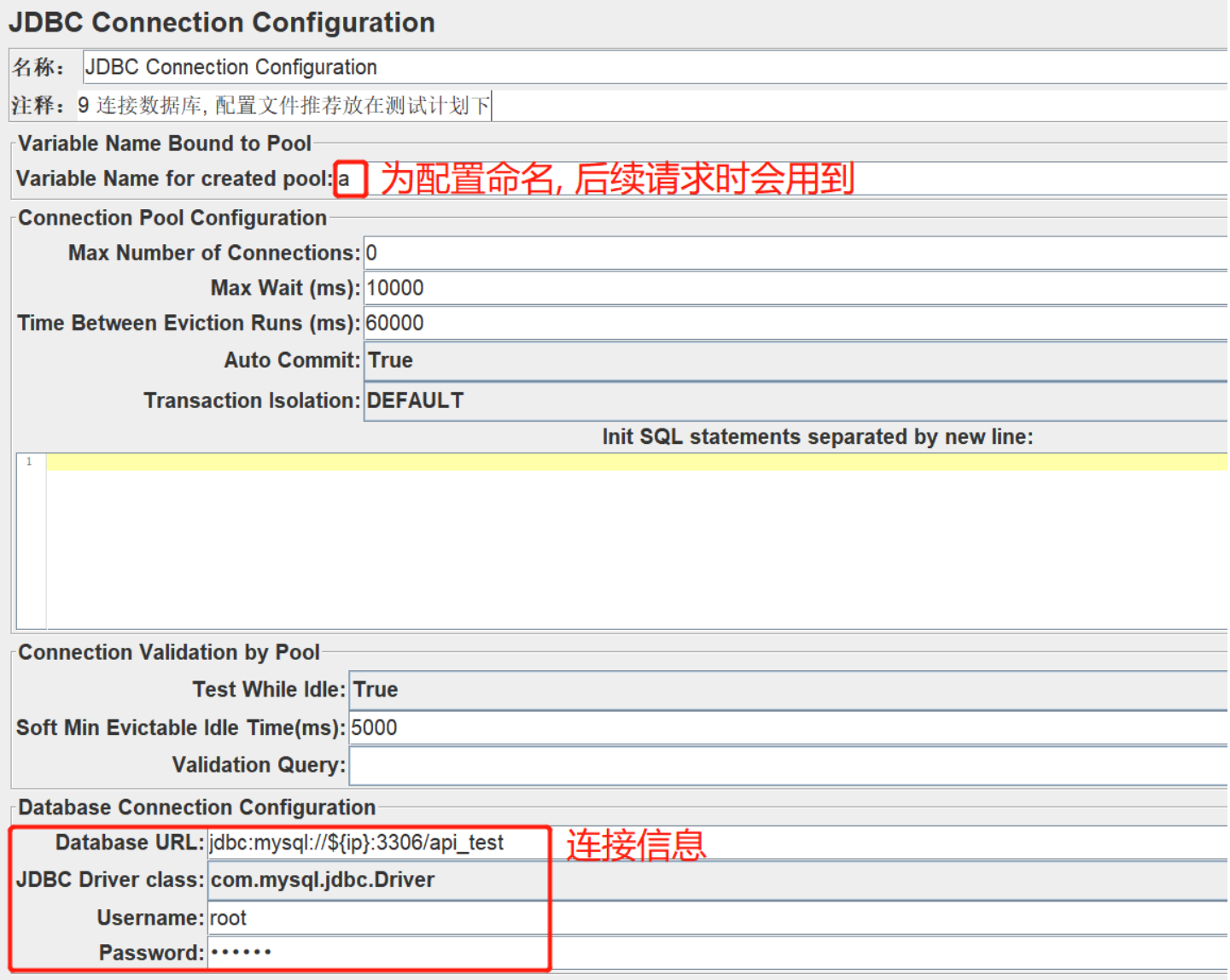
**实现步骤**

1. 导入第三方驱动包, 在测试计划下

> 驱动包的版本是向下兼容的, 也可以使用这个 mysql-connector-java-8.0.33.jar




2. 配置数据库连接信息 (配置元件中找到对应的组件)




3. 核心：向数据库发送SQL语句，并接收响应结果

**注意**

JDBC请求中, 返回的数据是一个列表, 如果你定义的变量名是 name, 那么引用结果的方式为 ${name_N}

- 假设我引用一个结果, 那么引用方式为 ${name_1}

## 多接口测试

**测试流程回顾**

```
代码块
1    1.需求分析,需求评审
```

```
2.编写测试计划
3.编写测试用例以及发起用例评审  --> 可以协助开发联调接口(通过性测试), 接口功能测试
4.[可选]搭建测试环境
5.系统测试,编写测试报告
        a.冒烟测试
        b.第一轮--新功能测试
        c.第二轮--bug回归
        d.第三轮--bug回归
        e....
        f.最后一轮旧功能回归  -->可以用接口回归测试来替代
6.验收测试,编写测试报告
7.线上,点点点
```

**常见的接口回归测试的策略**

- 基于模块, 回归每个模块的核心接口
  - 接口覆盖率高
  - 工作量较大, 项目迭代更安全
- 基于业务流, 回归核心业务流程
  - 接口覆盖率低
  - 工作量较少, 项目迭代更高效

> 可以看出, 不论是哪一种接口回归的策略, 都涉及多个接口, 也叫**多接口测试**

**多接口测试的原则**

- 接口覆盖率没必要100%, 只需要测试重要的接口
- 只需要设计正向的测试用例
- 脚本需要重复执行

## 10 依赖登录的接口测试

🔖 **背景:**

- **学习视频** `18 接口测试-多接口测试-10-依赖登录的接口测试-1` **中会提到api_server项目的启动和停止方式, 也简单提到了后续会把api_server项目整合到虚拟机中, 后续所有用到api_server的地方都使用虚拟机1里面的这个项目**

**目标: 轻松拥有属于自己的api_server项目**

**内容: 使用虚拟机1**

- 虚拟机1的应用5: api_server项目的启动和访问

## 10.1 项目和技术介绍

**项目介绍**

案例是api_server, 是一个电商系统的后端部分

- 启动: 项目根目录执行 `node app.js`
- 停止: Ctrl + C 即可

> 登录用户为admin/123456

**技术介绍-关联**

- 从API文档中发现问题, 请求其他接口前需要先登录获取token, 然后在访问其他接口时带上token
  - 类似的场景还有很多, 不只局限于需要登录, 如:
    - 测试考试交卷的业务流程, 需要学生先登录, 再进行交卷
    - 测试支付的流程, 需要先下单, 再进行支付
    - 修改学生信息, 需要先新增一个学生, 再进行修改
    - [**共同特点**] 调用下游接口前必须先调用上游接口
- 此类场景用到的技术就是**关联** (**概念: 提取上游接口的部分响应, 在请求下游接口时提交**)
- 关联的实现思路: **提取**上游接口的部分响应, 在请求下游接口时**引用**
- 关联的**作用**: 在不同接口间传递数据

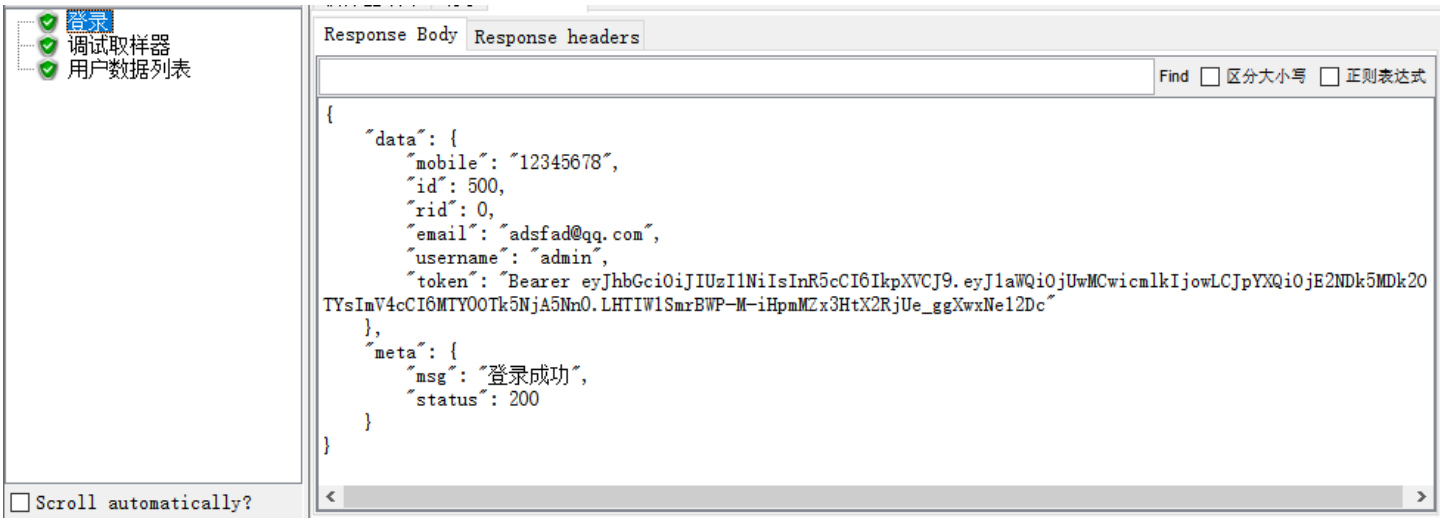
## 10.2 关联

**需求**

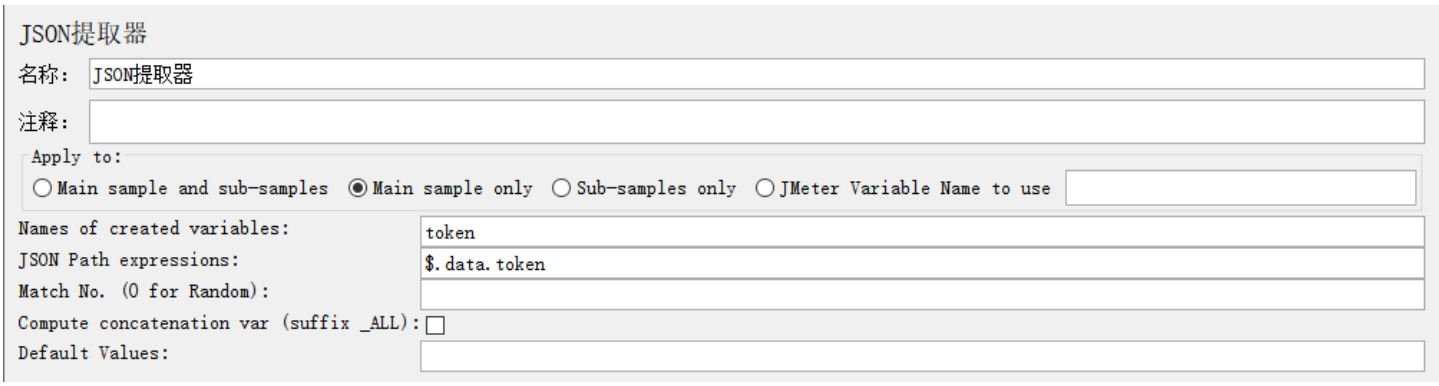
请求用户数据列表接口 (API文档中的1.3.1)

**步骤**

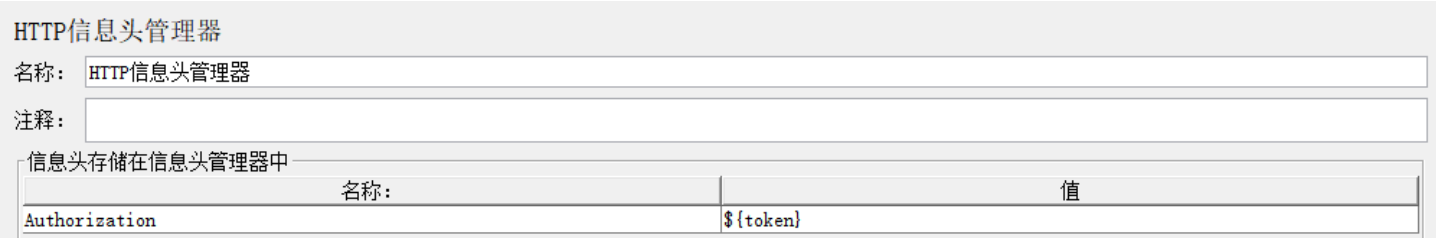
1. 脚本实现搭框架, 先访问登录接口, 获取token




2. 提取token (使用**JSON提取器**)




3. 携带token访问用户数据列表接口




## 10.3 跨线程组关联

### 需求

在之前案例的基础上, 把请求A和请求B设置到不同的线程组, 再实行一次

> - 结果: 请求B接口失败, 无法引用token
> - 原因: 变量作用域不对
> - 解决: 将局部变量转换成全局变量

### 步骤

1. 通过setProperty函数, 建立局部变量和全局变量的对应关系

2. 通过BeanShell取样器, 执行setProperty函数, 生成全局变量(属性)
3. 下游线程组通过Property函数引用全局变量即可

> 如何查看 jmeter 中的全部变量?
>
> 可以通过一个辅助性组件 属性显示: 测试计划 ---> 添加 ---> 非测试元件 ---> 属性显示

## 11 基于模块的多接口测试

**案例**

演示基于模块如何编写测试脚本, 对象为用户管理模块

> 产生的脚本文件为 `基于模块的多接口测试.jmx`

## 12 基于业务流的多接口测试

**业务概述**

业务指的是**谁**需要**处理什么事情**

- 软件中的业务指的是**谁**可以使用软件的**什么功能, 处理什么实际工作内容**

**业务流的含义**

指的是业务实现所经历整个流程, 也就是要实现某个业务, 需要操作的步骤

**业务流分析思路**

1. 对软件价值有**整体**认知
2. 找出所有角色, 以及每个角色能使用到哪些功能

**业务流分析结果**

- HR (管理员)
  - 核心业务
    - 新人入职
      - 管理员登录1.2.1 -> 添加用户1.3.2 -> 添加角色1.5.2 -> 角色授权1.5.6 -> 分配用户角色1.3.7

        > 最后清除冗余数据: 先删用户, 再删角色
    - 老员工离职
  - 分支业务

        - 员工人事调动, 跨岗位调动
        - 员工二进宫
        - ...
- 运营人员
    - 核心业务
        - 商品上架
            - 运营人员登录 -> 添加商品分类 -> 添加分类参数 -> 添加商品
        - 商品下架
    - 分支业务
        - 编辑商品, 换个分类
        - 上架后, 再下架, 再上架
        - ...
- 客服人员...

# Jmeter小结

## 文件上传问题

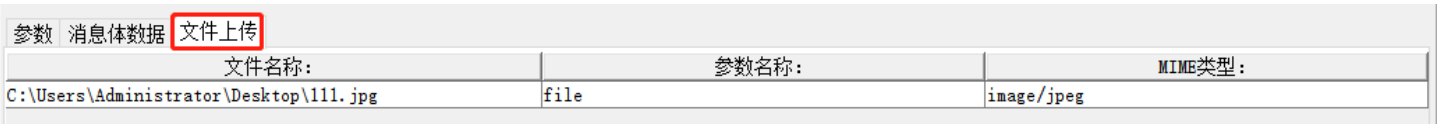
### 需求

测试电商后台API文档中图片上传接口1.9

> 只需要验证通过性即可

### 实现

HTTP请求中, 在 `文件上传` 中提交请求数据




> MIME: 是描述消息内容类型的因特网标准
>
> 参考: https://www.w3school.com.cn/media/media_mimeref.asp

### 面试题

接口测试过程中, 遇到文件上传的操作, 怎么处理?[2]

- 文件上传接口和其他接口一样, 无非是在传参时需要注意一下类型问题
- jmeter中有一个单独的窗口叫文件上传, 填写对应的参数名和文件路径后只需要修改一下MIME类型即可
- 如, 传的是图片, 填写image类型即可, 如, 传的是视频, 填写video类型即可

## 需要加密的接口测试

此处说的不是接口具备加密功能, 而是为了安全考虑, 请求的某些参数需要加密

- 如, 最常见的, 登录密码需加密
- 如, 支付信息需加密

### 常见加密方式了解

- MD5: 一种简单的哈希算法, 将任意长度数据通过算法生成一个固定长度的值, 速度快, 但安全性较差
- SHA-256: 比MD5更安全, 但也有安全隐患, 比之更安全的是 **SHA-256加盐**
- 此外还有AES, RSA等加密算法

### 以MD5加密为例, 说明jmeter如果解决需要对数据进行加密的问题

- jmeter中的内置函数
- 自己写加密代码, 在 **BeanShell** 中执行

代码块

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
public class ToBeCalled {
    public static String encryptMD5(String input) {
        try {
            // 创建MD5加密对象
            MessageDigest md = MessageDigest.getInstance("MD5");
            // 执行加密操作
            byte[] messageDigest = md.digest(input.getBytes());
            //将字节数组转换为16进制字符串
            StringBuilder hexString = new StringBuilder();
            for (byte b : messageDigest) {
                String hex = Integer.toHexString(0xff & b);
                if (hex.length() == 1) {
                    hexString.append('0');
                }
                hexString.append(hex);
```

```
            }
            // 返回加密后的字符串
            return hexString.toString();
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException(e);
        }
    }
}
//导出变量
vars.put("a", new ToBeCalled().encryptMD5("123456"))
```

- 也可以调用开发写的代码, 在 **BeanShell** 中调用

代码块

```
// 引入java文件
source("D:\\ToBeCalled.java");
// 调用方法（如加密方法）
String res = new ToBeCalled().encryptMD5("123456");
// 保存变量（可在借助调试取样器查看）
vars.put("rrr", res);
```

> BeanShell是一种轻量级Java脚本语言, 完全符合Java规范, 详情参考

**面试题**

遇到需要加密的接口该如何测试?[2]

1. 接口是否加密不影响我们的测试方法和流程
2. 如果遇到传参需要加密的接口, 我会首先确认加密方式
    - 如果是简单的MD5加密, 那么jmeter内置函数就可以实现
    - 如果是类似SHA-256加盐 这种复杂的加密方式, 则会让开发提供加密函数, 我直接在Bean Shell中调用即可

jmeter中的beanshell组件有没有用过, 什么时候用的?[1]

- 当然
- 最常见的使用场景是用来执行jmeter的内置函数, 如在跨线程组关联中需要把局部变量设置为全局的属性
- 另外在测试需要加密的接口时也会用到, beanshell来执行数据的加解密操作

## 第三方接口

### 应用场景

本公司产品中, 某些功能需要依赖第三方提供的服务, 如: 支付, 地图, 快递/物流查询等服务

### 面试题

依赖第三方接口的接口测试怎么做?[1]

- 接口中含有对第三方接口的请求, 这种情况很普通
- 从黑盒角度对接口进行测试和其他自研接口一样, 只需要做好字段验证, 业务逻辑和异常的验证即可
- 只不过在定位bug时, 需要定位清楚属于调用错误还是第三方接口的bug
  - 如果是自研代码错误, 则提交给本公司开发进行修复
  - 如果是第三方接口本身的问题, 则考虑反馈(对方更新后, 再重新测试) 或者更换第三方服务

## 作用域

### 概念

作用范围, 在jmeter中专指当前组件对哪些范围的**取样器**生效

### 分类

1. 取样器自身: 无所谓作用域, 在作用域概念中作为参考物存在
2. 大多数组件: 如, 查看结果数, http信息头管理器, csv数据文件设置, json提取器...
   - 测试计划下: 对所有线程组中的所有取样器生效
   - 线程组下: 对当前线程组中的所有取样器生效
   - 取样器下: 对当前取样器生效

## 执行顺序

jmeter 的组件有默认的执行顺序: (jmeter中组件的执行顺序与添加顺序无关)

代码块

```
    1) 配置元件(config elements): http请求默认值、信息头管理器、JDBC配置
       特点:配置初始化数据
    2) 前置处理器(Pre-processors): 用户参数
            特点:设置取样器执行所需数据
    3) 定时器(timers): 常量吞吐定时器、同步定时器
```

```
            特点:设置取样器的执行规则
    4) 取样器(Sampler): http请求、JDBC Request
            特点:核心
    5) 后置处理程序(Post-processors): JSON提取器
            特点:从响应结果提取数据
    6) 断言(Assertions): 响应断言
       特点:判断响应结果
    7) 监听器(Listeners): 查看结果树、聚合报告
            特点:显示最终的运行结果
```

## 图形化测试报告

jmeter支持以**图形化**的方式显示脚本的运行结果

### 应用场景

- 对测试报告有更高的要求时
- 执行性能测试时, 命令行执行的方式比图形化界面占用更少的资源

### 使用方式

代码块

```
# 生成图形化测试报告, 命令:
jmeter -n -t 脚本文件 -l 日志文件 -e -o 目录

-n: 无图形化界面的方式运行
-t: 指定被执行的脚本
-l: 结果写入的日志文件
-e: 生成测试报告
-o: 讲测试报告输出到某个目录
```

> 注意:
>
> 1. 执行命令时, 日志文件应不存在, 需执行后生成
> 2. 执行命令时, 报告输出目录应为空 或 不存在

### 面试题

接口测试报告包含哪些重点内容?[1]

1. 测试结果
2. 缺陷汇总和分析
3. 风险评估
4. 工作总结和改进

接口测试你是怎么做的? (接口测试流程)[3]

1. 首先确认需求和分工
2. 拿API文档, 了解具体接口的业务, url, 请求方式, 入参的各个字段, 以及响应数据
3. 设计接口测试用例, 并组织用例评审
    - 如果是功能测试: 要求100%覆盖新接口, 每个接口设计正向+逆向的测试用例
    - 如果是回归测试: 不要100%覆盖全接口, 只要求覆盖重要的和使用频率高的接口, 每个接口只需要1条正向的用例, 最后还要求脚本能重复执行
4. 用jmeter编写测试脚本, 执行, 并查看结果
5. 产出测试报告

如何使用jmeter来做接口测试?[3]

1. 首先拿到API文档, 了解接口业务, 包括接口地址, 请求方式, 入参, 出参, token鉴权, 返回格式等信息
2. 设计测试用例后, 使用jmeter工具执行测试, 一般步骤是这样的:
    a. 首先新建一个线程组
    b. 然后新建一个HTTP请求默认值 (输入服务器的IP和端口)
    c. 再创建HTTP请求, 一个取样器对应一个接口, 输入路径, 请求方式, 请求参数
    d. 然后创建断言和查看结果数
3. 最后是调试并执行用例
4. (当然用jmeter的过程中可能涉及一些技术问题, 比如响应数据乱码, 需要修改jmeter的配置文件, 编码方式改为UTF-8, 比如后续接口需要token, 我就会用到json提取器和beanshell来解决跨线程组关联的问题等等)